



VIT[®]
B H O P A L
www.vitbhopal.ac.in

CSE 2006 - Programming in Java

Course Type: LP

Credits: 3

Prepared by

Dr Komarasamy G

Senior Associate Professor

School of Computing Science and Engineering

VIT Bhopal University

Unit-2 Java Object-oriented Programming

- **Java OOP (Basics)**
- Java Class and Objects, Java Methods, Java Constructor, Java Strings, Java Access Modifiers, Java this keyword, Java final keyword, Java Recursion, Java instance of Operator, Java Single Class and Anonymous Class, Java enum Class
- **Java OOP (Inheritance & Polymorphism)**
- Java Inheritance, Java Method Overriding, Java super Keyword, Abstract Class & Method, Java Interfaces, Java Polymorphism (overloading & overriding), Java Encapsulation
- **Java OOP (Other types of classes)**
- **Nested & Inner Class, Java Static Class, Java Anonymous Class, Java Singleton, Java enum Class, Java enum Constructor, Java enum String, Java Reflection.**

Nested & Inner Class

- **Java inner class or nested class is a class that is declared inside the class or interface.**
- We use inner classes to logically group classes and interfaces in one place to be more readable and maintainable.
- Additionally, it can access all the members of the outer class, including private data members and methods.
- **Syntax of Inner class**

```
class Java_Outer_class
{
    //code
    class Java_Inner_class
    {
        //code
    }
}
```

Nested & Inner Class

- **Advantage of Java inner classes**

- There are three advantages of inner classes in Java. They are as follows:

1. Nested classes represent a particular type of relationship that is it can access all the members (data members and methods) of the outer class, including private.
2. Nested classes are used to develop more readable and maintainable code because it logically group classes and interfaces in one place only.
3. **Code Optimization:** It requires less code to write.

Nested & Inner Class

- **Need of Java Inner class**
- Sometimes users need to program a class in such a way so that no other class can access it. Therefore, it would be better if you include it within other classes.
- If all the class objects are a part of the outer object then it is easier to nest that class inside the outer class. That way all the outer class can access all the objects of the inner class.

Nested & Inner Class

- **Difference between nested class and inner class in Java**
- An inner class is a part of a nested class. Non-static nested classes are known as inner classes.

Types of Nested classes

- There are two types of nested classes non-static and static nested classes. The non-static nested classes are also known as inner classes.
- **Non-static nested class (inner class)**
 - Member inner class
 - Anonymous inner class
 - Local inner class
- **Static nested class**

Nested & Inner Class

Type	Description
Member Inner Class	A class created within class and outside method.
Anonymous Inner Class	A class created for implementing an interface or extending class. The java compiler decides its name.
Local Inner Class	A class was created within the method.
Static Nested Class	A static class was created within the class.
Nested Interface	An interface created within class or interface.

Nested & Inner Class

Java Member Inner class

A non-static class that is created inside a class but outside a method is called **member inner class**. It is also known as a **regular inner class**. It can be declared with access modifiers like public, default, private, and protected.

Syntax:

```
class Outer
{
    //code
    class Inner
    {
        //code
    }
}
```


Nested & Inner Class

Java Member Inner Class Example

In this example, we are creating a msg() method in the member inner class that is accessing the private data member of the outer class.

TestMemberOuter1.java

```
class TestMemberOuter1
{
    private int data=30;
    class Inner{
        void msg() {System.out.println("data is "+data);}
    }
    public static void main(String args[]){
        TestMemberOuter1 obj=new TestMemberOuter1();
        TestMemberOuter1.Inner in=obj.new Inner();
        in.msg();
    }
}
```

Output:
data is 30

Java Anonymous inner class

- **Java anonymous inner class** is an inner class without a name and for which only a single object is created.
- An anonymous inner class can be useful when making an instance of an object with certain "extras" such as overloading methods of a class or interface, without having to actually subclass a class.
- In simple words, a class that has no name is known as an anonymous inner class in Java. It should be used if you have to override a method of class or interface.

Java Anonymous inner class can be created in two ways:

1. Class (may be abstract or concrete).
2. Interface

Java Anonymous inner class

Java anonymous inner class example using class

TestAnonymousInner.java

```
abstract class Person{
    abstract void eat();
}
class TestAnonymousInner{
    public static void main(String args[]){
        Person p=new Person(){
            void eat(){System.out.println("nice fruits");}
        };
        p.eat();
    }
}
```

Output:
nice fruits

Java Anonymous inner class

Java anonymous inner class example using interface

```
interface Eatable{  
    void eat();  
}  
class TestAnnonymousInner1{  
    public static void main(String args[]){  
        Eatable e=new Eatable(){  
            public void eat(){System.out.println("nice fruits");}  
        };  
        e.eat();  
    }  
}
```

Output:
nice fruits

Java Anonymous inner class

Java Local inner class

- A class i.e., created inside a method, is called local inner class in java. Local Inner Classes are the inner classes that are defined inside a block. Generally, this block is a method body. Sometimes this block can be a for loop, or an if clause.
- Local Inner classes are not a member of any enclosing classes. They belong to the block they are defined within, due to which local inner classes cannot have any access modifiers associated with them. However, they can be marked as final or abstract. These classes have access to the fields of the class enclosing it.

If you want to invoke the methods of the local inner class, you must instantiate this class inside the method.

Java Anonymous inner class

Java local inner class example

LocalInner1.java

```
public class localInner1{  
    private int data=30;//instance variable  
    void display(){  
        class Local{  
            void msg(){System.out.println(data);}  
        }  
        Local l=new Local();  
        l.msg();  
    }  
    public static void main(String args[]){  
        localInner1 obj=new localInner1();  
        obj.display();  
    }  
}
```

Output:

30

Java static nested class

- A static class is a class that is created inside a class, is called a **static nested class in Java**. It cannot access non-static data members and methods. **It can be accessed by outer class name.**
- It can access static data members of the outer class, including private.
- The static nested class cannot access non-static (instance) data members.

Java static nested class example with instance method TestOuter1.java

```
class TestOuter1{
    static int data=30;
    static class Inner{
        void msg(){System.out.println("data is "+data);}
    }
    public static void main(String args[]){
        TestOuter1.Inner obj=new TestOuter1.Inner();
        obj.msg();
    }
}
```

Output:
data is 30

In this example, you need to create the instance of static nested class because it has instance method msg(). But you don't need to create the object of the Outer class because the nested class is static and static properties, methods, or classes can be accessed without an object.

Java Singleton class

- Singleton is a design pattern that ensures that a class can only have one object.
- To create a singleton class, a class must implement the following properties:
 - Create a private constructor of the class to restrict object creation outside of the class.
 - Create a private attribute of the class type that refers to the single object.
 - Create a public static method that allows us to create and access the object we created.
 - Inside the method, we will create a condition that restricts us from creating more than one object.

Java Singleton class

- **Example: Java Singleton Class Syntax**

```
class SingletonExample {  
    // private field that refers to the object  
    private static SingletonExample singleObject;  
    private SingletonExample()  
    { // constructor of the SingletonExample class }  
  
    public static SingletonExample getInstance()  
    { // write code that allows us to create only one object  
        // access the object as per our need  
    } }  
}
```

<https://www.programiz.com/java-programming/singleton>

Java Singleton class

- **In the above example,**
- private static **SingletonExample singleObject** - a reference to the object of the class.
- private **SingletonExample()** - a private constructor that restricts creating objects outside of the class.
- **public static SingletonExample getInstance()** - this method returns the reference to the only object of the class. Since the method static, it can be accessed using the class name.

Use of Singleton in Java

Singletons can be used while working with databases.

They can be used to create a connection pool to access the database while reusing the same connection for all the clients.

Java Singleton class

```
class Database {  
    private static Database dbObject;  
  
    private Database() {  
    }  
  
    public static Database getInstance() {  
        // create object if it's not already created  
        if(dbObject == null) {  
            dbObject = new Database();  
        }  
        // returns the singleton object  
        return dbObject;  
    }  
  
    public void getConnection() {  
        System.out.println("You are now connected to the database.");  
    }  
}
```

```
class Main {  
    public static void main(String[] args) {  
        Database db1;  
  
        // refers to the only object of Database  
        db1= Database.getInstance();  
  
        db1.getConnection();  
    }  
}
```

Output:

You are now connected to the database.

Java Singleton class

In our above example,

- We have created a singleton class Database.
- The dbObject is a class type field. This will refer to the object of the class Database.
- The private constructor Database() prevents object creation outside of the class.
- The static class type method getInstance() returns the instance of the class to the outside world.
- In the Main class, we have class type variable db1. We are calling getInstance() using db1 to get the only object of the Database.
- The method getConnection() can only be accessed using the object of the Database.
- Since the Database can have only one object, all the clients can access the database through a single connection.

Java enum Class

- In Java, an enum (short for enumeration) is a type that has a fixed set of constant values. We use the enum keyword to declare enums.
- For example,

enum Size

```
{  
    SMALL, MEDIUM, LARGE, EXTRALARGE  
}
```

Here, we have created an enum named Size. It contains fixed values SMALL, MEDIUM, LARGE, and EXTRALARGE.

These values inside the braces are called enum constants (values).

Note: The enum constants are usually represented in uppercase.

More: <https://www.programiz.com/java-programming/enums>

Java enum Class

- **Example 1: Java Enum**

```
enum Size {  
    SMALL, MEDIUM, LARGE, EXTRALARGE  
}  
  
class en {  
    public static void main(String[] args) {  
        System.out.println(Size.SMALL);  
        System.out.println(Size.MEDIUM);  
    }  
}
```

Output:
SMALL
MEDIUM

Java enum Constructor

- In Java, an enum class may include a constructor like a regular class. These enum constructors are either
- private - accessible within the class
- or**
- package-private - accessible within the package

Java enum Constructor

- **Example: enum Constructor**

```
enum Size {          // enum constants calling the enum constructors
    SMALL("The size is small."),
    MEDIUM("The size is medium."),
    LARGE("The size is large."),
    EXTRALARGE("The size is extra large.");
    private final String pizzaSize;
    // private enum constructor
    private Size(String pizzaSize) {
        this.pizzaSize = pizzaSize;
    }
    public String getSize() {
        return pizzaSize;
    }
}
```

```
class Main {
    public static void main(String[] args) {
        Size size = Size.SMALL;
        System.out.println(size.getSize());
    }
}
```

Output:

The size is small.

Java enum Constructor

- In the above example, we have created an enum Size. It includes a private enum constructor. The constructor takes a string value as a parameter and assigns value to the variable pizzaSize.
- Since the constructor is private, we cannot access it from outside the class. However, we can use enum constants to call the constructor.
- In the Main class, we assigned SMALL to an enum variable size. The constant SMALL then calls the constructor Size with string as an argument.
- Finally, we called getSize() using size.

Java enum String

- Before you learn about enum strings, make sure to know about Java enum.
- In Java, we can get the string representation of enum constants using the toString() method or the name() method. For example,

```
enum Size {  
    SMALL, MEDIUM, LARGE, EXTRALARGE  
}
```

```
class Main {  
    public static void main(String[] args) {
```

```
        System.out.println("string value of SMALL is " + Size.SMALL.toString());  
        System.out.println("string value of MEDIUM is " + Size.MEDIUM.name());
```

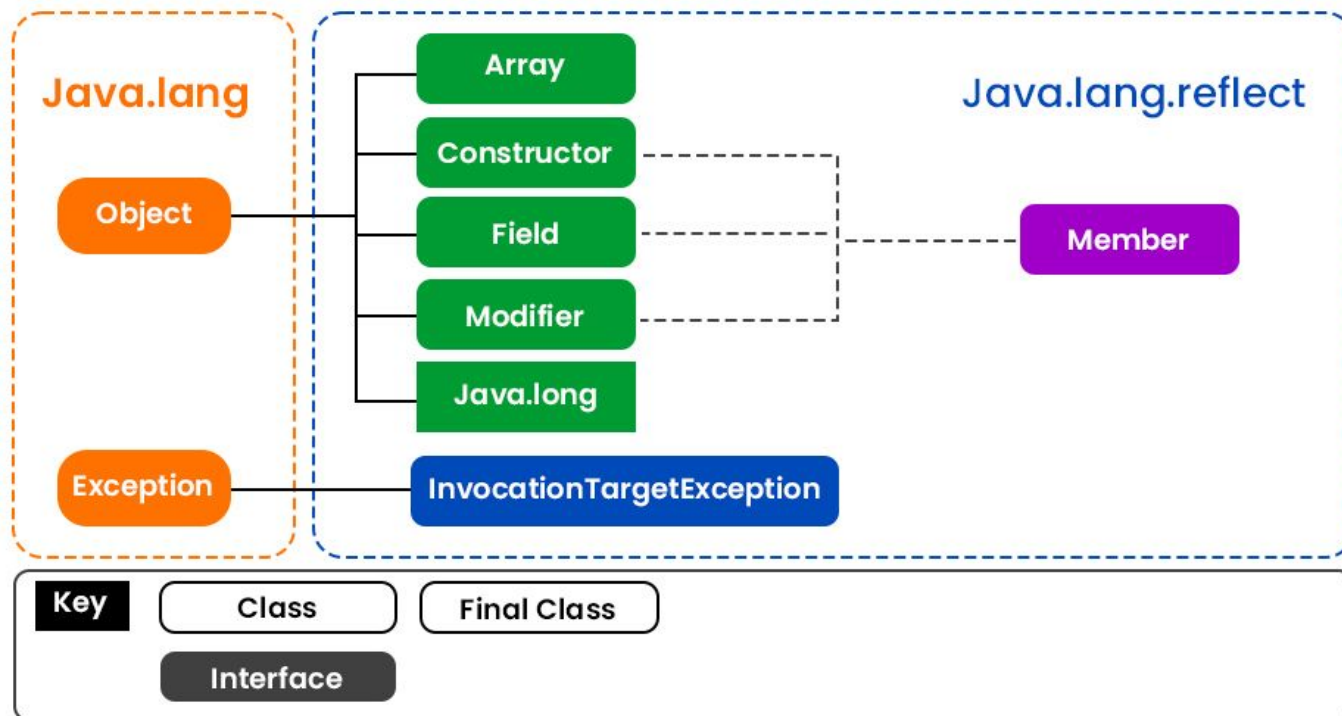
```
    }  
}
```

Output:

```
string value of SMALL is SMALL  
string value of MEDIUM is MEDIUM
```

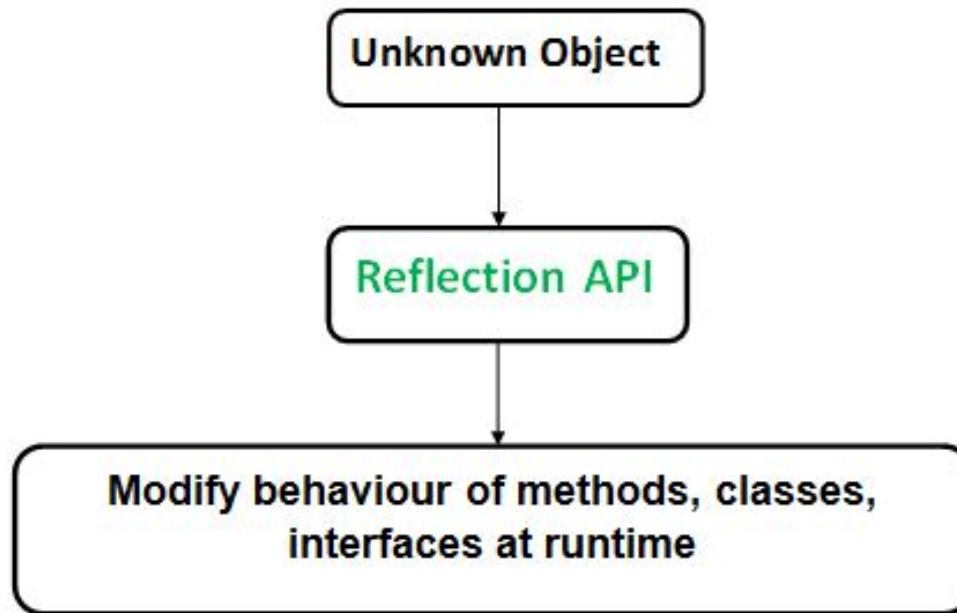
Java Reflection

- Reflection is an API that is used to **examine or modify the behavior of methods, classes, and interfaces at runtime.**
- The required classes for reflection are provided under **java.lang.reflect** package which is essential in order to understand reflection. So we are illustrating the package with visual aids to have a better understanding as follows:



Java Reflection

Reflection gives us information about the class to which an object belongs and also the methods of that class that can be executed by using the object. Through reflection, we can invoke methods at runtime irrespective of the access specifier used with them.



Java Reflection

Reflection can be used to get information about class, constructors, and methods as depicted below in tabular format as shown:

Class	The getClass() method is used to get the name of the class to which an object belongs.
Constructors	The getConstructors() method is used to get the public constructors of the class to which an object belongs.
Methods	The getMethods() method is used to get the public methods of the class to which an object belongs.

<https://www.geeksforgeeks.org/reflection-in-java/>

Java Reflection

- In Java, reflection allows us to inspect and manipulate classes, interfaces, constructors, methods, and fields at run time.
- There is a class in Java named Class that keeps all the information about objects and classes at runtime. The object of Class can be used to perform reflection.

Reflection of Java Classes

In order to reflect a Java class, we first need to create an object of Class.

And, using the object we can call various methods to get information about methods, fields, and constructors present in a class.

There exists three ways to create objects of Class:

1. Using `forName()` method

```
class Dog {...} // create object of Class to reflect the Dog  
class Class a = Class.forName("Dog");
```

Here, the `forName()` method takes the name of the class to be reflected as its argument.

2. Using `getClass()` method

```
// create an object of Dog class  
Dog d1 = new Dog(); // create an object of Class // to reflect Dog  
Class b = d1.getClass();
```

Here, we are using the object of the Dog class to create an object of Class.

3. Using `.class` extension

```
// create an object of Class // to reflect the Dog class  
Class c = Dog.class;
```

Now that we know how we can create objects of the Class. We can use this object to get information about the corresponding class at runtime.

Example: Java Class Reflection

```
import java.lang.Class;
import java.lang.reflect.*;
class Animal { }
// put this class in different Dog.java file
public class Dog extends Animal {
    public void display() {
        System.out.println("I am a dog.");
    }
}
// put this in Main.java file
```


Example: Java Class Reflection

// put this in Main.java file

```
class Main {  
    public static void main(String[] args) {  
        try {            // create an object of Dog  
            Dog d1 = new Dog();        // create an object of Class using getClass()  
            Class obj = d1.getClass();    // get name of the class  
            String name = obj.getName();  
            System.out.println("Name: " + name);  
            // get the access modifier of the class  
            int modifier = obj.getModifiers();  
            // convert the access modifier to string  
            String mod = Modifier.toString(modifier);  
            System.out.println("Modifier: " + mod);  
            // get the superclass of Dog  
            Class superClass = obj.getSuperclass();  
            System.out.println("Superclass: " + superClass.getName());  
        }  
        catch (Exception e) {  
            e.printStackTrace();  
        } } }
```

Output:

Name: Dog

Modifier: public

Superclass: Animal

<https://www.programiz.com/java-programming/reflection>